VITYARTHI PROJECT

(FUNDAMENTALS OF AI AND ML)

TOPIC:- AI-Driven Expert Medical Diagnostic System

Technology Stack:- Python 3.x, Tkinter, SWI prolog

Submitted By:- Aryan Saxena

Registration number:- 25BCY10045

University Mail Id:- aryan.25bcy10045@vitbhopal.ac.in

Submitted on:- 24 November 2025

INTRODUCTION

The Hybrid Expert Diagnostic System used in this project is intended to give a patient a two-pronged evaluation based on two important symptoms: body temperature and duration of symptoms. The system combines a Symbolic AI (Prologue) component for rule-based diagnosis and treatment recommendation with a Machine Learning (ML) component for probabilistic risk assessment.

The main objective is to show how these two AI paradigms work together: the transparency and logical inference of rule-based systems, and the pattern recognition and prediction power of machine learning.

Problem Statement

Both probabilistic risk assessment and precise, rule-based clinical guidance are frequently needed for modern diagnostics. The challenge is to develop a single, cohesive system that can:

1. Use straightforward numerical patient inputs (duration and temperature).

2. Give a quick, data-driven Risk Category evaluation (Low, Medium, or High).

3. Use a predetermined set of expert rules to deduce a conclusive diagnosis and a treatment plan.

4. Use a user-friendly graphical user interface (GUI) to display these findings.

Functional Requirements (FR)

ID	Requirement Description	Source/Component
FR1	The system shall accept Body Temperature (*C) and Symptom Duration (Days) as input.	app.py (GUI)
FR2	The system shall validate input to ensure temperature >=35*C and duration > 0.	app.py
FR3	The system shall load and train a Decision Tree classifier using symptoms_data.csv if the model file (risk_classifier.pkl) is missing.	app.py, classifier_model.py
FR4	The system shall use the trained ML model to predict a Risk Category (Low, Medium, High) for the given inputs.	classifier_model.py
FR5	The system shall translate numerical inputs into Prolog facts (fever_high/fever_low, duration_long/duration_short) based on defined thresholds.	app.py (Prolog Integration)
FR6	The Prolog engine shall use the asserted facts to determine a **Diagnosis** (e.g., severe_infection, mild_viral_fever).	rules.pl
FR7	The Prolog engine shall retrieve the corresponding **Treatment Plan** text for the determined diagnosis.	rules.pl
FR8	The GUI shall display the ML Risk Assessment and the Prolog Diagnosis and Treatment Plan.	app.py (GUI)

Non-functional Requirements (NFR)

ID	Requirement Description	Category
NFR1	**Performance:** The diagnosis and risk assessment should complete within 3 seconds of button press.	Time
NFR2	**Usability:** The Tkinter GUI should be simple and intuitive for a non-technical user (e.g., a nurse or a patient self-assessment).	User Interface
NFR3	**Reliability:** The Prolog engine must be initialized correctly, and input errors must be handled gracefully with clear messagebox alerts.	Error Handling
NFR4	**Maintainability:** The diagnostic logic (rules) should be easily editable in the separate rules.pl file without altering the main Python code.	Modularity
NFR5	**Portability:** The system requires external dependency on SWI-Prolog for the pyswip library to function.	Environment

System Architecture

The system uses a hybrid architecture that combines AI components that are knowledge-driven and data-driven.

1. Presentation Layer (app.py): User input is received and results are displayed by the Tkinter GUI.

2. Controller/Integration Layer (app.py): Serves as a bridge by contacting the ML service and using pyswip to communicate with the Prologue engine.

3. Machine Learning Service (classifier_model.py): In charge of training, storing, loading, and executing forecasts for Risk Category evaluation.

4-Expert System Engine (Prologue): The runtime inference engine (pyswip/SWI-Prolog) and knowledge base (rules.pl) utilised for symbolic diagnosis and treatment lookup.

Design Diagrams

To ensure that developers, testers, and product owners all have a common understanding of the application's architecture and behaviour, system modelling diagrams are essential.

Use Case Diagram

- **Primary Actor:** User/Patient
- **Use Cases:**
  - Enter Symptoms (Temperature, Duration)
  - Run AI Analysis
  - View Risk Assessment (ML Result)
  - View Diagnosis & Treatment (Prolog Result)

Workflow Diagram

Step	Component	Description
1. Start	User/GUI (app.py)	The user enters **Body Temperature** and **Symptom Duration** and clicks the "Run AI Analysis & Diagnosis" button.
2. Input Validation	app.py	The system checks if inputs are valid numbers and satisfy conditions(T>=35*C , D>0).
3. ML Risk Assessment	classifier_model.py	Python calls load_and_predict_risk(temp, duration). The loaded Decision Tree model predicts the **Risk Category** (Low, Medium, or High) based on the input data.
4. Display Risk	app.py	The GUI updates the Risk Assessment label, changing the color based on the category (e.g., Red for High, Green for Low).
5. Prolog Fact Setup	app.py	Python retracts old facts and asserts new facts (symptom/2) into the Prolog knowledge base based on thresholds.
6. Prolog Diagnosis Query	app.py	Python executes the query prolog.query("get_diagnosis(X, T)").
7. Rule Inference	rules.pl (Prolog Engine)	Prolog uses the asserted facts to satisfy the diagnosis/1 rules (e.g., severe_infection, mild_viral_fever). The get_diagnosis/2 rule links the diagnosis to the corresponding treatment/2 text.
8. Retrieve Results	app.py	Python receives the inferred **Diagnosis** and **Treatment Plan** text.
9. Display Results	app.py	The GUI updates the Diagnosis and Treatment labels with the results from the rule base.
10. End	System	The analysis is complete.

Sequence Diagram

Participant	Message
User	Enters Temp/Duration
:app.py (GUI)	← temp, duration
:app.py	Calls load_and_predict_risk(temp, duration)
:classifier_model.py	Loads Model (risk_classifier.pkl)
:classifier_model.py	→ risk_category
:app.py	Updates GUI (Risk Label)
:app.py	prolog.retractall(...)
:app.py	prolog.assertz(symptom(patient, ...))
:app.py	prolog.query("get_diagnosis(X, T)")
:Prolog Engine	Performs Inference
:Prolog Engine	→ diagnosis, treatment
:app.py	Updates GUI (Diagnosis/Treatment)

Class/Component Diagram

- **DiagnosticApp (app.py):** Main controller, handles GUI, I/O, and integration.
  - *Methods:* run_diagnosis(), GUI setup.

- **ClassifierModel (classifier_model.py):** Handles data and ML logic.
  - *Methods:* load_data_and_train(), load_and_predict_risk().

- **PrologEngine (rules.pl):** Defines the knowledge base.
  - *Predicates:* symptom/2, diagnosis/1, treatment/2, get_diagnosis/2.

ER Diagram

The system does **not** use a relational database for persistent storage. The knowledge base is stored in the static Prolog file (rules.pl) and the data is loaded from a CSV file (symptoms_data.csv) for model training.

Design Decisions & Rationale

Decision	Rationale
Hybrid Approach (ML + Prolog)	Uses ML for fast, probabilistic **risk** prediction, and Prolog for transparent, logical **diagnosis** based on expert-defined rules.
Tkinter for GUI	Tkinter is lightweight, built-in to Python, and sufficient for this simple prototype, avoiding extra dependencies like PyQt or Kivy.
Decision Tree Classifier (ML)	Simple, fast to train on small datasets, and its decision logic is relatively interpretable, aligning with the rule-based theme.
Prolog Fact Assertion in Python	The Python code preprocesses the continuous numerical data into discrete symbolic **symptoms** (fever_high, duration_long) before querying Prolog, simplifying the rule logic in rules.pl.

Implementation Details

→ **Language:** Python 3.x for the application logic and GUI, Prolog for the expert knowledge base.

→ **Libraries:** tkinter, pyswip, scikit-learn, numpy, joblib.

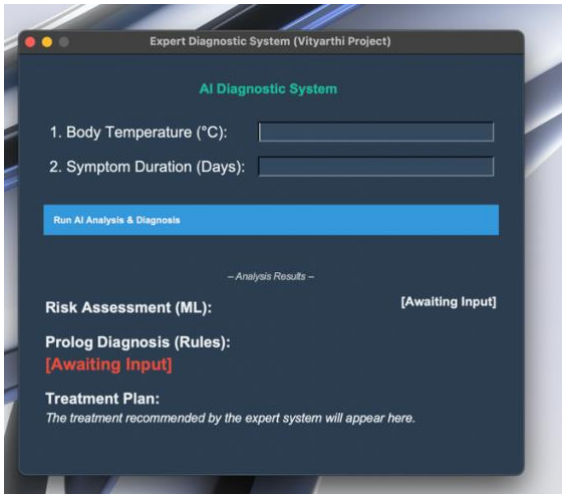
→ **Model Training:** The classifier_model.py module handles model lifecycle. On the first run, app.py triggers load_data_and_train(), which saves the model to risk_classifier.pkl.

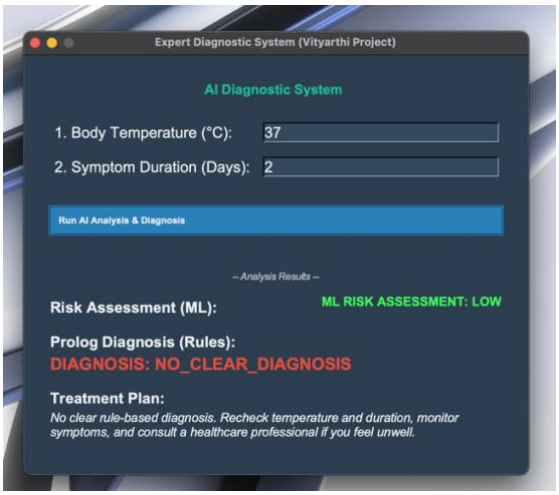
→ **Thresholds:**

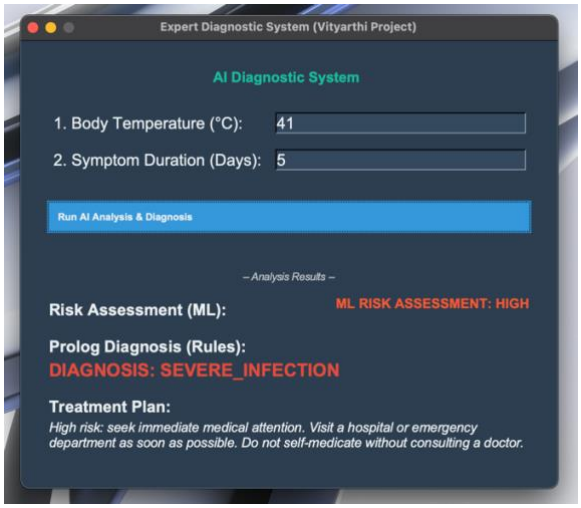
- **High Fever:** Temp >= 39.0*C.
- **Low Fever:** 37.5*C <= TEMP < 39.0*C
- **Long Duration:** Duration >= 5 days

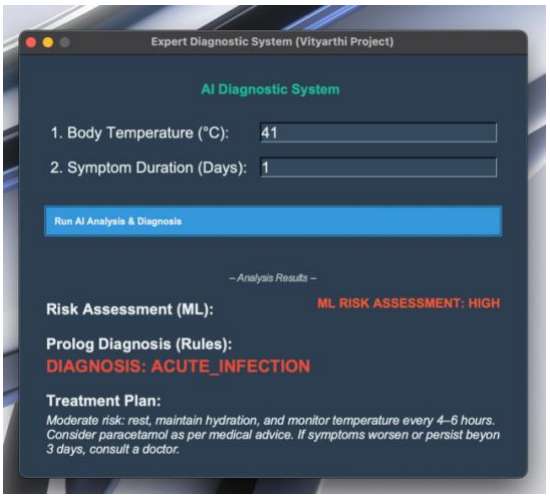
Screenshots / Results

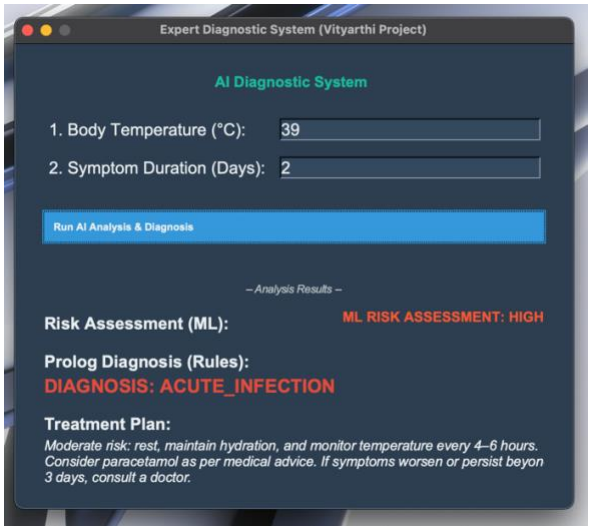
(WITH USER INTERFACE)

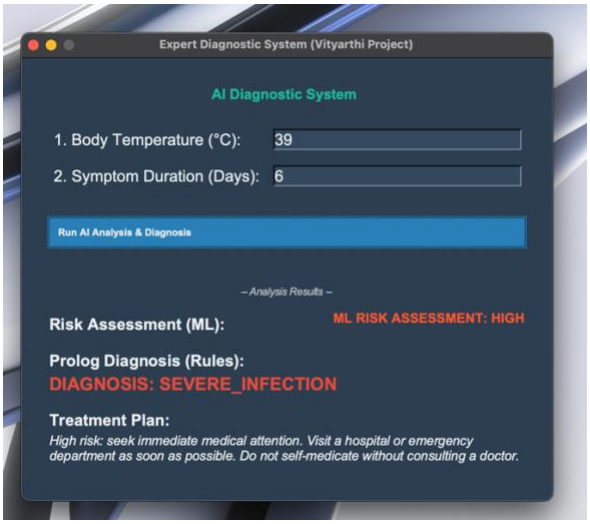


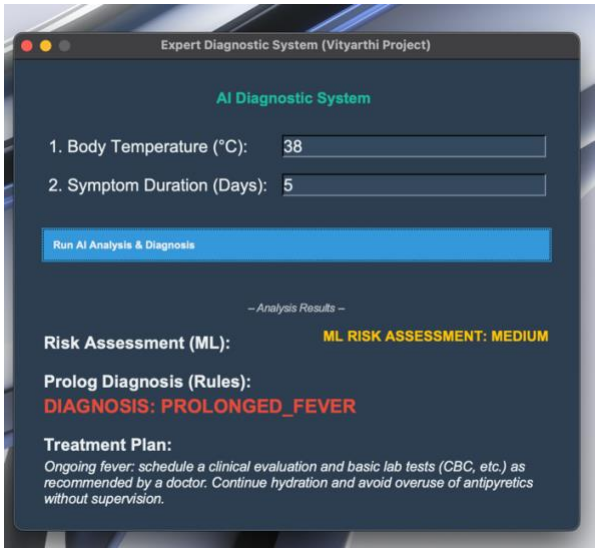


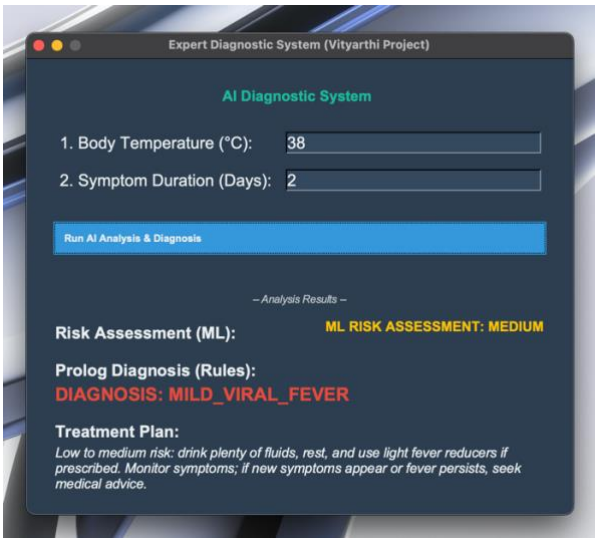





Expert Diagnostic System (Vityarthi Project)
AI Diagnostic System
1. Body Temperature (°C): 39
2. Symptom Duration (Days): 2
Run AI Analysis & Diagnosis
– Analysis Results –
Risk Assessment (ML):
ML RISK ASSESSMENT: HIGH
Prolog Diagnosis (Rules):
DIAGNOSIS: ACUTE_INFECTION
Treatment Plan:
Moderate risk: rest, maintain hydration, and monitor temperature every 4–6 hours. Consider paracetamol as per medical advice. If symptoms worsen or persist beyon 3 days, consult a doctor.


Expert Diagnostic System (Vityarthi Project)
AI Diagnostic System
1. Body Temperature (°C): 39
2. Symptom Duration (Days): 6
Run AI Analysis & Diagnosis
– Analysis Results –
Risk Assessment (ML):
ML RISK ASSESSMENT: HIGH
Prolog Diagnosis (Rules):
DIAGNOSIS: SEVERE_INFECTION
Treatment Plan:
High risk: seek immediate medical attention. Visit a hospital or emergency department as soon as possible. Do not self-medicate without consulting a doctor.


Expert Diagnostic System (Vityarthi Project)
AI Diagnostic System
1. Body Temperature (°C): 38
2. Symptom Duration (Days): 5
Run AI Analysis & Diagnosis
– Analysis Results –
Risk Assessment (ML):
ML RISK ASSESSMENT: MEDIUM
Prolog Diagnosis (Rules):
DIAGNOSIS: PROLONGED_FEVER
Treatment Plan:
Ongoing fever: schedule a clinical evaluation and basic lab tests (CBC, etc.) as recommended by a doctor. Continue hydration and avoid overuse of antipyretics without supervision.


Expert Diagnostic System (Vityarthi Project)
AI Diagnostic System
1. Body Temperature (°C): 38
2. Symptom Duration (Days): 2
Run AI Analysis & Diagnosis
– Analysis Results –
Risk Assessment (ML):
ML RISK ASSESSMENT: MEDIUM
Prolog Diagnosis (Rules):
DIAGNOSIS: MILD_VIRAL_FEVER
Treatment Plan:
Low to medium risk: drink plenty of fluids, rest, and use light fever reducers if prescribed. Monitor symptoms; if new symptoms appear or fever persists, seek medical advice.

Testing Approach

Testing was primarily conducted using **unit testing** and **scenario testing**.

- **ML Testing:** Validated that the Decision Tree correctly predicts risk categories for boundary conditions in symptoms_data.csv. For example, 37.0*C, 1 day → Low; 39.5*C, 6 days → High.
- **Prolog Rule Testing:** Tested all four major diagnostic scenarios by simulating fact assertion from the Python side:
    1. High Fever + Long Duration → severe_infection.
    2. High Fever + Short Duration → acute_infection.
    3. Low Fever + Short Duration → mild_viral_fever.
    4. Low Fever + Long Duration → prolonged_fever.
- **Integration Testing:** Confirmed that the app.py successfully translates input, interacts with both the ML model and Prolog, and displays the correct combined output.

Challenges Faced

→ **Prolog Initialization:** Setting up the pyswip library required correct installation and path configuration for the external SWI-Prolog engine, which can be an external environment dependency issue.

→ **Data Translation:** The challenge was defining the correct thresholds (>= 39.0*C for fever_high, >= 5 days for duration_long) to effectively map continuous input data to discrete Prolog facts without loss of critical information.

→ **Styling in Tkinter:** Achieving a modern, consistent look (using the dark DiagnosticTheme) required manipulating the ttk.Style module, which is often less flexible than modern web frameworks.

Learnings & Key Takeaways

1. AI Hybridisation: Discovered the usefulness of integrating various AI approaches to address a single issue, such as using ML for data-driven risk assessment and symbolic AI for clear, rational decision-making (diagnosis).

2-Prolog Integration: Acquired expertise in dynamically asserting and retracting facts from a host language (Python) in a Prologue knowledge base, which is essential for creating stateful expert systems.

3. Model Deployment: Recognised the fundamental procedures for deploying a minimal machine learning model, which include training, saving (joblib), and loading for inference in a primary application.

Future Enhancements

1. Data and Model Complexity: For the risk assessment, use a larger, real-world dataset and a more sophisticated machine learning model (such as SVM or Logistic Regression).

2. Rich Symptom Input: To make the rule base more comprehensive, expand the input to include discrete symptoms (such as cough, headache, and fatigue) that would be directly asserted as Prologue facts.

3. Uncertainty Handling: Instead of depending solely on true/false rule matching, use fuzzy logic or certainty factors in Prologue to handle uncertain diagnoses.

4. Web/Mobile Deployment: For greater usability and accessibility, move the Tkinter GUI to a web framework (such as Flask or Django).

References

1:- Vityarthi Modules

2:- Language Reference for Python (https://docs.python.org/3/)

3:- SWI Prolog Official Documentation

4:- Tkinter Official Documentation (Widget parameters and methods reference)

5:- MVC Design Patterns and Object-Oriented Programming (OOP) (Conceptual framework)

6:- Documentation for the Tkinter Font Module (tkinter.font used for custom typography)